

Getting PyTorch to Actually Use Your RTX 5090: A Complete WSL2 Setup Guide for Blackwell (sm_120)



Getnet Demil

Follow

8 min read · Mar 30, 2026



51



From a broken Windows pip install to a fully working PyTorch 2.12 build on CUDA 12.9 inside WSL2 — every failure, every fix, and every command.



The Problem Nobody Warns You About

You buy an RTX 5090. You install PyTorch. You run your training script. It works — sort of. The GPU shows up in `nvidia-smi`, tensors move to CUDA without errors, and you feel like everything is fine.

It is not fine.

What is actually happening is that PyTorch is falling back to a compatibility mode that does not use the Blackwell architecture properly. The RTX 5090 is a Compute Capability 12.0 GPU (`sm_120`). Every stable PyTorch release as of early 2026 was compiled for architectures up to `sm_90` (Hopper). When PyTorch encounters `sm_120` it either raises a warning and partially degrades, or throws:

CUDA error: no kernel image is available for execution on the device

This means the compiled CUDA kernels in the PyTorch wheel have no binary for your GPU. The GPU is present but fundamentally unusable for deep learning operations.

This post documents the complete journey from that error to a working PyTorch 2.12.0 build with full sm_120 support, compiled from source inside WSL2 on Ubuntu 24.04 with CUDA 12.9.

Why the Windows Path Fails

The natural first instinct is to fix this on Windows, where most researchers keep their projects. There are three approaches on Windows and all of them fail:

Approach 1: Standard pip install

The stable PyTorch wheel (cu124) does not include sm_120 kernels. You get the silent degradation or the kernel image error described above.

Approach 2: PyTorch nightly cu135

PyTorch nightly builds for Linux include sm_120 support. Windows wheels for cu135 and cu136 are not published. The index URL exists but returns no matching wheel for Windows:

ERROR: Could not find a version that satisfies the requirement torch

Approach 3: Community builds (cu128.nv)

A custom Hugging Face build (torch-2.6.0+cu128.nv) does work on Windows and was the working solution for a period. However it is an unofficial build,

not maintained long term, and introduces its own DLL dependency issues on machines without the right Visual C++ redistributables installed.

The conclusion is straightforward: building PyTorch with proper sm_120 support on Windows is not a supported path. WSL2 is.

Setting Up WSL2

WSL2 (Windows Subsystem for Linux version 2) runs a real Linux kernel inside a lightweight VM on Windows. NVIDIA's drivers expose the GPU through to WSL2 via a passthrough layer. From Linux's perspective you have a real CUDA-capable GPU.

Prerequisites

- Windows 11 or Windows 10 build 19041+
- NVIDIA driver 525.85 or newer on Windows (the Linux driver inside WSL2 is provided automatically)
- WSL2 enabled (not WSL1)

Open WSL2

Press **Win + R**, type `wsl`, press Enter. Or open Windows Terminal and select Ubuntu from the dropdown.

You land as root. Always switch to your user account immediately:

```
su — yourusername
```

Verify the GPU is visible inside WSL2:

```
nvidia-smi
```

You should see your RTX 5090 listed with Driver Version from Windows (e.g. 576.88) and CUDA Version 12.9. If this fails, your WSL2 GPU passthrough is not configured — ensure you are running WSL2 not WSL1, and that your NVIDIA driver version is recent enough.

The CUDA Toolkit Trap on Ubuntu 24.04

This is the most important section because it is where most builds silently fail or produce a GPU that appears to work but does not.

Ubuntu 24.04's default apt repository includes a package called `nvidia-cuda-toolkit`. Do not install it. It installs CUDA 12.0, which predates Blackwell support entirely. `nvcc` from this package will reject `sm_120`:

```
nvcc fatal : Unsupported gpu architecture 'compute_120'
```

The second trap is NCCL. If you previously installed NCCL from NVIDIA's repo, you may find a version like `libnccl2 2.29.7-1+cuda13.2` sitting on your system while your toolkit is CUDA 12.0 or 12.6. This version mismatch causes linker errors deep in the build that are difficult to diagnose.

The Correct CUDA Installation Path

Remove any existing CUDA toolkit installed via apt:

```
sudo apt remove -purge -y nvidia-cuda-toolkit nvidia-cuda-dev nvidia-cuda-gdb \ nvidia-cuda-toolkit-doc libcudart12 sudo apt autoremove -y
```

Add NVIDIA's official CUDA repository for Ubuntu 24.04:

```
wget  
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86\_64/  
cuda-keyring\_1.1-1\_all.deb sudo dpkg -i cuda-keyring_1.1-1_all.deb sudo apt  
update
```

Install CUDA 12.9 (the minimum version that supports sm_120 in nvcc and aligns with available NCCL packages):

```
sudo apt install -y cuda-toolkit-12-9
```

Add CUDA 12.9 to your PATH. Append these to ~/.bashrc:

```
export PATH=/usr/local/cuda-12.9/bin:$PATH export  
LD_LIBRARY_PATH=/usr/local/cuda-12.9/lib64:$LD_LIBRARY_PATH
```

Reload and verify:

```
source ~/.bashrc nvcc -version
```

Expected output:

```
Cuda compilation tools, release 12.9, V12.9.86
```

System Build Dependencies

Install the full development stack. Most of these will already be present on a fresh Ubuntu 24.04:

```
sudo apt install -y build-essential cmake git curl wget ninja-build \ python3-  
dev python3-pip python3-venv \ libopenblas-dev libblas-dev liblapack-dev \  
libjpeg-dev libpng-dev gcc g++ gfortran
```

For NCCL, install the version matching CUDA 12.9:

```
sudo apt install -y libnccl2 libnccl-dev
```

Note: if you are building for single-GPU use only (which is the common case for most researchers), you can skip NCCL entirely by setting `USE_NCCL=0` during the build. This significantly reduces build complexity and is the recommended approach unless you are running multi-GPU distributed training.

Creating the Python Environment

Use a plain Python venv rather than conda for the build. Conda introduces its own library paths that can conflict with the system CUDA toolkit during compilation:

```
python3 -m venv ~/torch5090 source ~/torch5090/bin/activate pip install -upgrade  
pip setuptools wheel ninja cmake
```

If you have conda installed (Miniconda or Anaconda), it will re-activate its base environment every time you source `.bashrc`. Deactivate it explicitly after activating your venv:

```
source ~/torch5090/bin/activate conda deactivate
```

Your prompt should show only (torch5090), not (torch5090) (base).

Cloning PyTorch

Clone the main branch with all submodules. The recursive clone is not optional — PyTorch depends on dozens of third-party libraries (FBGEMM, XNNPACK, pybind11, protobuf, etc.) that live as git submodules:

```
git clone -recursive https://github.com/pytorch/pytorch ~/pytorch cd ~/pytorch git
submodule sync git submodule update -init -recursive
```

The submodule sync step updates the submodule URLs in case any have moved since the last commit. Without it, submodule updates can silently fail and leave you with empty third-party directories that cause missing header errors deep in the build.

Get Getnet Demil's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Install Python build requirements inside your venv:

```
pip install -r requirements.txt
```

Build Environment Variables

These variables control exactly what gets compiled. Set them all before starting the build:

```
export CUDA_HOME=/usr/local/cuda-12.9 export
PATH=$CUDA_HOME/bin:$PATH export
```



```
LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH export  
TORCH_CUDA_ARCH_LIST="12.0" export CMAKE_PREFIX_PATH=$(python -c  
'import sysconfig; print(sysconfig.get_paths()["purelib"])') export USE_CUDA=1  
export USE_NCCL=0 export USE_DISTRIBUTED=0 export USE_CUDNN=1  
export USE_FLASH_ATTENTION=0 export USE_MEM_EFF_ATTENTION=0  
export MAX_JOBS=4
```

The critical variables:

- —1 tells nvcc to compile kernels specifically for sm_120. Without this, the build will either compile for a default list that excludes sm_120, or try to compile for every architecture and massively increase build time.
- —2 flash attention compiles hundreds of CUDA kernel variants and is extremely memory hungry. Each nvcc invocation can consume 3–4 GB RAM. With MAX_JOBS=4 and flash attention enabled you easily hit 16+ GB just from compiler processes, causing OOM kills (exit code 137).
- —3 limits parallel ninja jobs. Counter-intuitively, more jobs causes OOM kills on CUDA compilation. 4 jobs with 7–8 GB each is stable on a 30 GB system. Adjust based on your available RAM.
- —4 disables multi-GPU communication libraries. For single-GPU workloads these add build time and complexity with no benefit.

Building PyTorch

Clean any previous failed build artifacts first. Stale CMake cache from a previous attempt against a different CUDA version will cause the new build to silently use wrong paths:

```
python setup.py clean rm -rf build
```

Open in app 

Sign up

Sign in

Medium

 Search

 Write



```
nohup pip install -v -no-build-isolation . > ~/pytorch_build.log 2>&1 & echo  
"Build PID: $!"
```

Monitor progress in real time:

```
tail -f ~/pytorch_build.log
```

Check if it is still running:

```
ps aux | grep pip
```

If the build fails, get the actual error rather than the generic pip error message:

```
grep -i "error\\|fatal\\|failed" ~/pytorch_build.log | grep -v "^-" | tail -30
```

Common Build Failures and Their Causes

Exit code 137 on CUDA files

OOM kill. The Linux kernel is killing individual nvcc processes because they consume too much RAM. Reduce MAX_JOBS and set USE_FLASH_ATTENTION=0.

nvcc fatal: Unsupported gpu architecture 'compute_120'

Your CUDA toolkit is too old. CUDA 12.8 is the minimum for sm_120 support. CUDA 12.9 is recommended.

NCCL linker errors

Version mismatch between libnccl and your CUDA toolkit. Either install NCCL built for your exact CUDA version, or set USE_NCCL=0.

Missing headers in third_party/

Submodule initialization was incomplete. Run `git submodule update -init -recursive` again from inside `~/pytorch`.

Terminated (no error message)

The WSL session was closed or the machine went to sleep. This is why `nohup` is essential. Restart the build — CMake will resume from where it stopped if you do not clean.

Verifying the Build

A successful build ends with:

Successfully built torch Successfully installed torch-2.12.0a0+gitfd1d1b0

Verify that sm_120 kernels are actually being executed:

```
python -c "import torch print('PyTorch version:', torch.__version__) print('CUDA available:', torch.cuda.is_available()) print('CUDA version:', torch.version.cuda) print('GPU:', torch.cuda.get_device_name(0)) print('Compute capability:', torch.cuda.get_device_capability(0)) a = torch.randn(1024, 1024, device='cuda') b
```

```
= torch.randn(1024, 1024, device='cuda') c = torch.matmul(a, b)
torch.cuda.synchronize() print('Matrix multiply: PASSED') "
```

Expected output:

```
PyTorch version: 2.12.0a0+gitfd1d1b0 CUDA available: True CUDA version: 12.9
GPU: NVIDIA GeForce RTX 5090 Compute capability: (12, 0) Matrix multiply:
PASSED
```

Compute capability (12, 0) confirms sm_120. The matrix multiply executing without error confirms the Blackwell kernels are present and running.

Daily Workflow

Add a shell alias so activation is a single command. Append to ~/.bashrc:

```
alias torch-gpu='source ~/torch5090/bin/activate && conda deactivate'
```

Set your default WSL user so WSL opens directly without requiring su -. Run this once in Windows PowerShell:

```
ubuntu config -default-user yourusername
```

Your complete daily startup becomes:

```
# Open WSL terminal torch-gpu

# activate environment cd ~/your-project

# go to project code . OR cursor .
```

For the editor to connect properly to WSL, always open it from inside WSL using code `.` or cursor `.`, not by opening a Windows folder path. This ensures the editor's terminal, Python interpreter, and file system all operate inside WSL with native I/O performance rather than crossing the Windows/WSL filesystem boundary.

Summary: What Actually Matters

CUDA version: 12.8 minimum, 12.9 recommended. CUDA 12.0 (Ubuntu apt default) and CUDA 12.6 do not support sm_120.

NCCL: Skip it (`USE_NCCL=0`) for single-GPU setups. Version mismatches are a common build failure source.

Flash attention: Disable during build (`USE_FLASH_ATTENTION=0`) to avoid OOM kills. Add it back separately once the base build works.

MAX_JOBS: 4 is safe for 30 GB RAM. More parallel jobs causes OOM kills on CUDA compilation.

Submodules: Always use `-recursive` clone and run `submodule sync + update`. Missing submodules cause cryptic missing header errors.

Build artifacts: Always clean (`python setup.py clean && rm -rf build`) before retrying after a failed build against a different CUDA version.

The RTX 5090 is a genuinely powerful GPU for research workloads. The setup friction is real but it is a one-time cost. Once the build is done, everything

downstream — torchvision, huggingface, custom CUDA extensions — works normally against this installation.

Nvidia Gpu

Rtx5090

Wsl



Written by Getnet Demil

18 followers · 109 following

Follow



Image Processing and Computer Vision, Communication System Engineer,
Electrical Engineer, web: getnetdemil.github.io

No responses yet



Write a response

What are your thoughts?